

Change Impact Predictor: Identifying Critical Test Cases After Function Signature Changes

Dyson Huo

huodx@ucla.edu

University of California, Los Angeles
Los Angeles, California, USA

Zhengli Shang

z4shang@ucla.edu

University of California, Los Angeles
Los Angeles, California, USA

Zijie Zhang

zjiezhang@ucla.edu

University of California, Los Angeles
Los Angeles, California, USA

Abstract

Function signature changes in large Python projects can silently break dependent tests, yet identifying the most affected tests remains difficult at scale. We present Change-Impact-Predictor, a lightweight tool that maps signature edits to impacted test files and ranks them by predicted importance. Our approach mines commit histories to detect signature changes via a fast RegEx-based diff miner and a more robust AST-based miner, then discovers candidate tests using efficient `git grep` patterns and applies a composite ranking model using frequency, structural depth, and test type signals. Evaluations on *pandas* and *scikit-learn* show strong ranking performance (MRR up to 0.94, Recall@10 up to 99.2%, Recall@5 up to 98.8%).

1 Introduction

Software systems evolve continuously, and as codebases grow, the coupling between components creates a complex web of dependencies. When developers modify function signatures by changing parameter types, return types, or adding and removing parameters, the effects can propagate throughout the system. Test files that depend on the modified interfaces may break or require updates to accommodate the new signatures. Identifying these affected test files is critical for maintaining software quality during evolution.

However, for large-scale projects such as *scikit-learn* and *pandas*, which contain thousands of test files spanning hundreds of thousands of lines of code, manually reviewing every test file to determine which ones are affected by a signature change is simply not feasible. Such manual inspection is both time-consuming and error-prone. A developer modifying a core utility function may inadvertently overlook test files in distant parts of the codebase that happen to depend on that function. Missing these dependencies can lead to test failures discovered late in the development cycle or, worse, bugs that escape into production.

Despite the practical importance of this problem, there is a notable gap in existing tools and academic studies specifically addressing the identification of test files affected by function signature changes. While the broader area of change impact analysis has received attention, most existing approaches focus on general code dependency tracking rather than the specific challenge of connecting signature modifications to their test suite implications. Furthermore, many existing

techniques rely on heavyweight static or dynamic analysis that may not scale well to large Python codebases with their dynamic typing and flexible calling conventions. In this work, we present Change Impact Predictor, a lightweight and practical tool for identifying critical test cases after function signature changes in Python projects. Our approach combines three key techniques:

1. AST based signature change detection: We parse Python source code using Abstract Syntax Trees to precisely identify changes in function parameter type annotations and return types. Unlike simple text based diffing, AST analysis can distinguish between semantically significant changes and superficial formatting modifications.
2. Efficient test dependency discovery: We leverage `git grep` with carefully crafted regular expression patterns to rapidly identify test files that reference changed functions. This search strategy uses word boundaries to ensure precise matches (for example, searching for `svd` does not match `svdvals`) while covering various usage patterns including direct calls, imports, qualified names, and string references common in test parameterization.
3. Impact based test ranking: We introduce a scoring system that ranks both test files and parameter changes by their predicted impact. Test importance is computed based on multiple factors including frequency of coupling with changed code, position in the test hierarchy where shallower tests are weighted higher, and test category where API and integration tests receive higher scores than narrow unit tests.

We evaluate our tool on two widely used open source Python libraries: *scikit-learn* and *pandas*. Our analysis mines thousands of commits from these repositories, extracting function signature changes along with their associated test dependencies. The results demonstrate that our approach effectively identifies and ranks impacted tests with high accuracy.

On *scikit-learn*, our method achieves strong ranking performance with a Mean Reciprocal Rank (MRR) of 0.9356 and Recall@1 of 89.6%, with Recall@5 and Recall@10 reaching 98.8% and 99.2% respectively.

On *pandas*, performance achieves an MRR of 0.5712 and Recall@1 of 38.8%, with Recall@5 at 73.5% and Recall@10 at

83.7%. These results indicate that relevant tests consistently appear at or near the top of our ranked recommendations, enabling developers to identify impacted tests with minimal inspection effort. Furthermore, our ranking correctly prioritizes high impact changes: for example, the highest ranked signature change in pandas (the `array` function) affects 409 test files, demonstrating how widely used core APIs naturally rise to the top of our rankings.

The remainder of this paper is organized as follows. Section 2 discusses related work in change impact analysis and test prioritization. Section 3 describes our methodology in detail, including the signature change detection algorithm, test discovery strategy, and ranking system. Section 4 presents our evaluation results on `scikit-learn` and `pandas`. Section 5 discusses the implications of our findings, threats to validity, and additional experiments. Section 6 concludes and outlines directions for future work.

2 Related Works

2.1 Code Refactoring Analysis

Understanding how code evolves through refactoring is a well studied area in software engineering. ActRef [2] is an action based refactoring analysis framework designed to detect and understand refactoring activities in Python codebases. By analyzing code change actions, ActRef identifies various refactoring types such as move, rename, extract, and inline operations across multiple granularity levels including variables, methods, classes, and modules. Their evaluation on 1,914 manually validated refactoring instances from 136 open source Python projects demonstrated high precision (0.80) and recall (0.92), outperforming existing baselines in detecting Python refactorings.

While ActRef shares our use of AST based analysis for understanding code changes, its focus differs fundamentally from ours. ActRef concentrates on categorizing and understanding refactoring patterns without directly considering how these changes affect associated test suites. In contrast, our work specifically targets the relationship between function signature modifications and test files. We aim not to classify the type of change, but rather to identify which test files are affected and may require updates or attention after a signature modification occurs.

2.2 Behavioral Change Detection

ChangeGuard [1] introduces a learning guided execution approach to validate code changes by comparing the runtime behavior of modified functions. It employs pairwise learning guided execution to detect unintended behavioral changes, particularly those introduced by automated code transformations. Applied to a dataset of 224 manually annotated code changes from popular Python open source projects, ChangeGuard identified semantics changing code changes with a precision of 77.1% and a recall of 69.5%.

ChangeGuard represents a different paradigm from our approach. It uses machine learning techniques combined with dynamic execution to determine whether a code change introduces unintended behavior. This requires generating and executing test inputs to observe runtime differences between old and new function versions. Our approach, by contrast, operates entirely through static analysis of source code and repository history. We do not attempt to determine whether a change is semantically correct or introduces bugs. Instead, we focus on the more targeted problem of identifying which test files reference changed functions and therefore may need developer attention. This makes our approach lightweight and scalable to large codebases where dynamic execution of all potentially affected code paths would be prohibitively expensive.

2.3 Regression Test Selection and Prioritization

The broader field of regression test selection has produced numerous techniques for identifying which tests to run after code changes. Classical approaches rely on static analysis of call graphs and control flow to trace dependencies between source code and tests. Dynamic approaches instrument program execution to record which tests exercise which code paths. These techniques have been extensively studied for languages like Java and C, where static type information and compilation dependencies provide clear linkages between code units.

However, Python presents unique challenges for change impact analysis. Its dynamic typing, duck typing conventions, and flexible import system make traditional static analysis more difficult to apply. Functions can be called through reflection, passed as first class objects, and referenced through computed attribute access, all of which complicate dependency tracking. Most existing tools for Python either rely on expensive dynamic analysis or provide only coarse grained dependency information at the module level.

2.4 Our Contribution

Our work addresses a gap not directly tackled by existing approaches. We focus specifically on function signature changes, which include modifications to parameter type annotations, return type annotations, and parameter lists. These changes are particularly important because they represent contract modifications that are likely to affect callers of the function. Unlike ActRef, we do not aim to classify the type of refactoring but rather to identify test files that may be impacted. Unlike ChangeGuard, we do not require runtime execution or machine learning models, making our approach more lightweight and immediately applicable to large repositories.

Our key differentiators are:

1. We provide direct mapping from function signature changes to affected test files, whereas ActRef focuses

on refactoring classification and ChangeGuard focuses on behavioral validation.

2. We use efficient repository mining with git grep for test discovery, avoiding the overhead of full static analysis or dynamic execution.
3. We introduce an impact based ranking system that helps developers prioritize which changes and tests require the most attention.
4. Our approach is specifically designed for Python code-bases and handles the challenges of dynamic typing through pattern based search strategies that account for various import and usage patterns.

3 Methods

3.1 RegEx-Based Signature Change Miner

We implement a regular expression (RegEx) driven static miner to efficiently detect function signature changes from Git commit histories. The miner operates directly on unified diffs and focuses on identifying parameter type annotation modifications, which are the primary source of downstream test breakages in typed Python code.

3.1.1 Change Detection.

For each commit, we parse its unified diff and scan only added and removed function definition lines, identified by the `+def` and `-def` prefixes. Two regular expressions are used:

- `SIG_LINE` extracts the function name, full parameter list, and optional return type annotation from a function definition line.
- `TYPE_ANN` extracts individual type annotations from parameter lists.

For every commit, we construct two maps:

- a removed-signature map from deleted lines (`-def`),
- an added-signature map from added lines (`+def`).

A function is flagged as having a signature change if it appears in both maps and its extracted parameter type annotations differ between the two versions. This ensures that we detect true type-level signature changes, rather than formatting or whitespace edits.

3.1.2 Commit Prefiltering.

To improve scalability, we apply a lightweight commit-level prefilter before full diff parsing. Specifically, we use `git log -G` with patterns indicative of signature changes, including:

- parameter type annotations,
- return type annotations,
- default parameter values.

This step filters out commits that do not contain any potential signature edits, significantly reducing the number of diffs that require detailed analysis.

3.1.3 Test Discovery.

For each function identified as changed, we perform a repository-wide test search using `git grep`. The search includes:

- direct function calls,
- method invocations,
- string references,
- and import statements.

When the function’s module path is known, we additionally search for module-qualified references. Test files are identified using both directory-based and filename-based heuristics, including:

- files under `test/` or `tests/` directories,
- files named `test_*.py`,
- files named `*_test.py`.

This approach allows us to conservatively recover all tests that may be impacted by a given signature change.

3.1.4 Parallelized Processing.

To scale to large repositories with thousands of commits, the miner executes commit-level analysis in parallel using Python’s `ProcessPoolExecutor`. The number of worker processes is set to CPU cores minus one, ensuring efficient utilization of available hardware while avoiding resource contention. This parallelization yields substantial speedups over sequential processing.

3.2 AST-Based Change Miner

To improve from the RegEx implementation baseline, we further implemented an abstract syntax tree (AST) based change miner. This miner still uses Git commands to provide versioned code implementation comparison, but then uses Python’s native `ast` package for AST parsing and a custom change-mining phase. This approach provides us with a more robust framework for performing change analysis and mining, overcoming the noise originating from literal but not structural differences in the source code.

3.2.1 AST Function Signature Change Detection

Similar to the RegEx version, we utilize git commands to retrieve two versions of a file across a commit. Then, the source codes of the old version and new version are parsed into AST trees and sent into an extractor, following the visitor pattern, for function information extraction. This visitor `FunctionVisitor` extracts nesting class hierarchy, function signature, and function body.

The `FunctionVisitor` class traverses the AST, keeping track of the current class context to generate fully qualified names for class methods (e.g., `ClassA::ClassB::method_name`). It uses the utility function `get_annotation_string` to convert the AST nodes representing type annotations back into their string representation (e.g., `List[str]`, `int`).

If an argument lacks an explicit annotation, its type is recorded as "Any". The extracted information for each function includes the argument names and their types, the return type, and a list of AST statement nodes representing the function body.

Then, we extract the common function names across the new and the old version, and hence find an intersection, regarding them as the functions which potentially could contain changes. We compare the extracted parameter type lists ($\text{old}[\text{'args'}]$ vs $\text{new}[\text{'args'}]$) and return types ($\text{old}[\text{'return'}]$ vs $\text{new}[\text{'return'}]$).

If the function body has not changed (match_bodies returns True) and the new argument list is a subtype of the old argument list ($\text{is_arglist_subtype}$ returns True), the change is ignored, as it suggests a type specialization that does not require a change in calling tests.

3.2.2 Rule-based Lightweight Type Inference

Python's type annotation mechanism proposes a major challenge to type change detection: the annotation is entirely optional. This means that it is possible for some repositories to contain function signature with only limited or even entirely missing type annotations. Therefore, it is also possible (and likely in practice) to first omit type annotations and then introduce them later versions.

In order to handle this issue, we propose a rule-based lightweight type inference mechanism that handles related scenarios under practical assumptions.

Our core assumption regarding function version updates is that if a function body capable of handling type T remains unchanged, it can inherently handle any subtype $T' <: T$. This principle generalizes to functions with multiple arguments by treating the argument list as a tuple. Furthermore, it accounts for unspecified argument types by treating them as Any, given that $\forall T, T' <: \text{Any}$. Formally, for an update $f_{\text{old}}(A) \rightarrow f_{\text{new}}(A')$:

$$\text{body}(f_{\text{old}}) = \text{body}(f_{\text{new}}) \wedge A' <: A \implies \text{no impact}$$

By simple derivation:

$$\forall i \in \{1, \dots, n\}, t_i <: t'_i \implies (t_1, t_2, \dots, t_n) <: (t'_1, t'_2, \dots, t'_n)$$

Therefore, for function with multiple arguments, $f(a_1 : T_1, a_2 : T_2, \dots, a_n : T_n)$, we consider tuple type that consists of $T = (T_1, T_2, \dots, T_n)$. If the new version's argument list type $T' <: T$, and the function body remains the same, we do not need to change tests for this function.

For now, we only added the **Any** subtype derivation, but this method can be smoothly expand to include more subtyping rules such as $\text{int} <: \text{int}|\text{float}$ etc.

3.2.3 API/Signature Change Taxonomy

To further gain insight into our approach, we developed a straightforward taxonomy for API/function signature changes; this provides a direct report for how, in the actual project

source code repositories, the API changes are distributed. Table 1 shows the categories we detect.

Table 1. API/Signature Change Categories

Change Type	Description	Example
ARG_ADD	Function adds an argument.	$\text{Foo}(a:\text{int}) \Rightarrow \text{Foo}(a:\text{int}, b:\text{int})$
ARG_DEL	Function removes an argument.	$\text{Foo}(a:\text{int}, b:\text{int}) \Rightarrow \text{Foo}(a:\text{int})$
ARG_ORD	Function argument order changed.	$\text{Foo}(a:\text{int}, b:\text{int}) \Rightarrow \text{Foo}(b:\text{int}, a:\text{int})$
TYPE_EXP	Function signature's type expanded (generalization).	$\text{Foo}(a:\text{int}) \Rightarrow \text{Foo}(a:\text{int} \text{float})$
BODY_MOD	Function signature has shrunk, but function body modified (type inference inconclusive).	$\text{Foo}(a:\text{Any}) \Rightarrow \text{Foo}(a:\text{dict}); \text{ Foo's body updated}$
OTHER	Other cases that do not fit perfectly into this taxonomy.	$\text{Foo}(a:\text{int}, b:\text{int}) \Rightarrow \text{Foo}(c:\text{str})$

The reason we have the category OTHER is that there could be potentially more complex changes happening in the same commit.

3.3 Test Ranking Method

For each test identified as potentially affected by a signature change, we assign a composite priority score based on three complementary signals: change frequency, structural depth, and test type. These signals capture how strongly a test is coupled to recent changes, how broadly it may affect the system, and how much functional surface it exercises.

3.3.1 Frequency Score

The frequency score measures how many distinct signature changes reference a given test across the analyzed commit window. Intuitively, if a test is repeatedly associated with multiple parameter changes, it is more likely to be affected by future modifications. We therefore treat higher frequency as a stronger indicator of failure risk.

3.3.2 Structural Depth Score

The structural depth score captures how deeply a test is nested within the project's package and directory hierarchy. Tests that are located closer to the root of the test structure tend to serve as shared integration points, while deeply nested tests are often narrowly scoped. We therefore assign

higher impact to shallower tests, as their failures typically reflect broader behavioral regressions.

3.3.3 Test Type Score

The test type score prioritizes tests by their functional scope. In particular, API and integration tests receive higher weight than unit tests, since they exercise larger portions of the execution pipeline and are more sensitive to interface-level changes. Unit tests, while precise, tend to be more localized and are thus weighted lower.

3.3.4 Composite Scoring and Change-Level Ranking

We combine the three signals into a single ranking score using a weighted linear model:

- 40% weight on Frequency,
- 30% weight on Structural Depth,
- 30% weight on Test Type.

For a given code change, we aggregate the scores of all tests associated with that change to produce an overall change impact score. Finally, we rank all changes by this impact score, allowing developers to focus first on the changes most likely to break the test suite.

We next apply this composite scoring scheme to all detected signature changes and rank them by their aggregated impact scores.

Table 2. Top-5 ranked signature changes on *pandas* based on impact score.

Rank	Score	Commit	Function	#Tests
1	24139	757e1518	array	409
2	21632	52272450	array	353
3	20758	64d544c7	array	336
4	20582	51c6e797	date_range	325
5	20507	1a82659e	array	331

Table 2 reports the top-ranked signature changes identified by our ranking model on *pandas*. The highest-impact changes are dominated by updates to `array` (four of the top five entries), each touching hundreds of tests (336–409), highlighting how foundational construction utilities can induce broad downstream impact across the test suite. The remaining entry, `date_range`, also triggers substantial impact (325 tests), consistent with its role as a widely used time-series API. Overall, these results suggest that our ranking method prioritizes signature changes to core, high-traffic APIs that propagate widely through dependent code and tests, helping developers focus first on the most disruptive updates.

Figure 1 presents the pipeline for the lightweight RegEx-based signature change miner (Section 3.1). Figure 2 presents the pipeline for the more robust AST-based signature change miner (Section 3.2). Finally, Figure 3 illustrates the composite

Figure 1: RegEx-Based Signature Change Miner

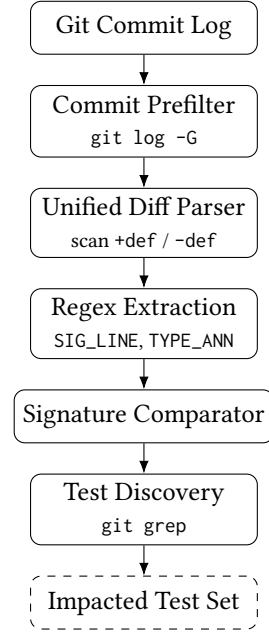


Figure 1. System pipeline for detecting function signature changes using the lightweight RegEx-based miner.

test ranking workflow (Section 3.3) common to both miners, showing how impacted tests are scored and ranked for developer inspection.

4 Evaluation Results

4.1 Evaluation Methodology

We evaluate the effectiveness of our test ranking approach through historical validation on real-world commit histories. For each repository, we replay past commits, apply our signature change miner and ranking pipeline, and compare the ranked test lists produced by our system against the tests that developers actually modified in those commits. This setup allows us to measure how well our method anticipates real maintenance behavior under realistic development conditions.

For every commit with detected signature changes, we generate a ranked list of predicted impacted tests. The ground truth for evaluation consists of the set of test files that were truly modified by developers in that same commit. We then assess how closely our ranked predictions align with this ground truth using two standard ranking-based metrics: Recall@K and Mean Reciprocal Rank (MRR).

Figure 2: AST-Based Signature Change Miner

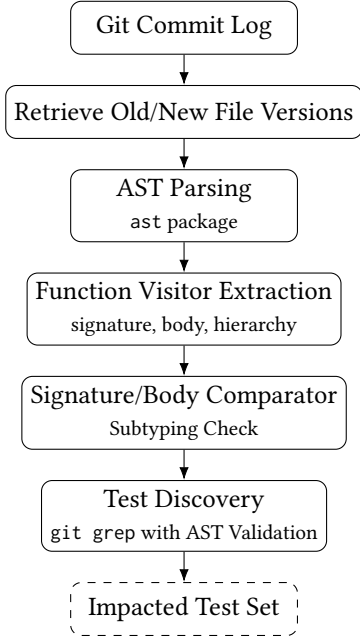


Figure 2. System pipeline for detecting function signature changes using the robust AST-based miner.

Figure 3: Composite Test Ranker Workflow

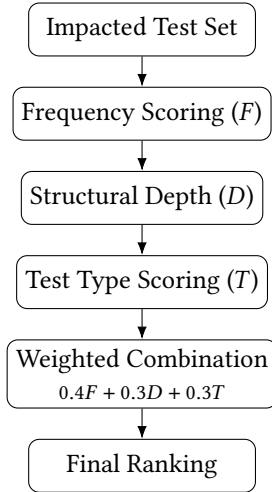


Figure 3. The composite ranking workflow applied to the impacted test set to prioritize developer attention.

4.1.1 Recall@K

Recall@K measures how many of the truly modified tests are successfully recovered within the top-K ranked predictions. It is defined as:

$$\text{Recall@K} = \frac{\#(\text{relevant tests in top-}K)}{\#(\text{total relevant tests})}$$

Recall@K captures the coverage of our recommendations: a high Recall@K indicates that by inspecting only the top-K suggested tests, developers can already recover most of the tests they would eventually modify.

4.1.2 Mean Reciprocal Rank (MRR)

MRR measures how highly the first relevant test appears in the ranked prediction list. For a single commit, we locate the rank of the first actually modified test and compute its reciprocal rank as:

$$\text{RR} = \frac{1}{\text{rank}}$$

The final MRR score is obtained by averaging the reciprocal ranks over all evaluated commits:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$$

where N is the total number of evaluated commits.

MRR therefore reflects the early precision of the ranking: a higher MRR indicates that relevant tests tend to appear very near the top of the recommendation list.

4.1.3 Evaluation Protocol

All evaluations are conducted on two large, real-world open-source repositories: *pandas* and *scikit-learn*. These projects are widely used, actively maintained, and contain thousands of historical commits and test cases, making them representative of practical, large-scale development environments. By evaluating on these repositories, we ensure that our results reflect realistic workloads rather than synthetic or small-scale benchmarks.

Based on the evaluation protocol described above, we present the ranking performance of our approach on two large real-world repositories. The detailed results are shown in Tables 3–5.

Table 3. Evaluation results on the *pandas* repository.

Metric	Value
Verified Commits	49
MRR	0.5712
Recall@1	38.8%
Recall@5	73.5%
Recall@10	83.7%

Tables 3–5 report the evaluation results on *pandas* and *scikit-learn* under the STRICT setting (i.e., verified call-site

Table 4. Evaluation results on the *scikit-learn* repository.

Metric	Value
Verified Commits	511
MRR	0.9356
Recall@1	89.6%
Recall@5	98.8%
Recall@10	99.2%

Table 5. Cross-repository comparison of ranking performance.

Repo	MRR	R@1	R@5	R@10
pandas	0.5712	38.8%	73.5%	83.7%
scikit-learn	0.9356	89.6%	98.8%	99.2%

modified test changes). On *scikit-learn*, our approach achieves strong ranking performance with an MRR of 0.9356 and Recall@1 of 89.6%, meaning the first relevant test is ranked at the top position for the vast majority of commits. Recall@5 and Recall@10 further reach 98.8% and 99.2%, indicating that nearly all ground-truth modified tests are recovered within the top few recommendations.

On *pandas*, performance is notably lower, with MRR = 0.5712 and Recall@1 = 38.8%. Recall improves as the cutoff increases (Recall@5 = 73.5%, Recall@10 = 83.7%), suggesting that relevant tests are often retrieved but tend to appear deeper in the ranked list compared to *scikit-learn*.

Overall, the results reveal a clear cross-repository gap: the method ranks impacted tests very effectively for *scikit-learn*, while *pandas* presents a more challenging setting where ranking is less sharp and requires larger cutoffs to achieve higher coverage. This highlights that repository-specific characteristics can materially affect ranking quality, and that practical usage in harder projects may benefit from inspecting a larger top-*k* recommendation set.

4.1.4 Distribution of Change Types

Table 6. Distribution of change types in the *pandas* repository.

Change Type	Count	Freq (%)
OTHER	4783	60.12
BODY_MOD	1818	22.85
ARG_DEL	724	9.10
ARG_ADD	610	7.67
TYPE_EXP	18	0.23
ARG_ORD	3	0.04
Total	7956	100.00

For *pandas* (Table 6), OTHER accounts for the majority of changes (60.12%), followed by BODY_MOD (22.85%). Compared

Table 7. Distribution of change types in the *scikit-learn* repository.

Change Type	Count	Freq (%)
ARG_ADD	674	44.46
ARG_DEL	590	38.92
OTHER	185	12.20
BODY_MOD	66	4.35
TYPE_EXP	1	0.07
ARG_ORD	0	0.00
Total	1516	100.00

to argument-level edits (ARG_ADD/ARG_DEL), this suggests that changes in *pandas* are more frequently non-signature modifications captured by our taxonomy.

For *scikit-learn* (Table 7), argument-level edits dominate: ARG_ADD and ARG_DEL together account for 83.38% of changes. This indicates that the sampled changes in *scikit-learn* are more concentrated in API/signature-level evolution compared to *pandas*.

Table 8. Cross-repository comparison of change-type distributions (frequency in %).

Change Type	pandas (%)	scikit-learn (%)
ARG_ADD	7.67	44.46
ARG_DEL	9.10	38.92
ARG_ORD	0.04	0
BODY_MOD	22.85	4.35
TYPE_EXP	0.23	0.07
OTHER	60.12	12.20

5 Discussion

5.1 Strengths of Our Approach

Lightweight and Scalable: Our approach relies entirely on static analysis of source code and efficient repository queries using git grep. Unlike dynamic analysis approaches that require program execution, or machine learning approaches that require training data and model inference, our method can analyze thousands of commits in minutes using parallel processing. This makes it practical for integration into continuous integration pipelines or developer workflows where quick feedback is essential.

Precise AST Based Signature Detection: We use Python’s Abstract Syntax Tree module to parse and compare function signatures at the syntactic level. Unlike simple text based diffing or regex matching, AST analysis can precisely identify parameter names, type annotations, and return types while ignoring irrelevant formatting changes. The AST approach also naturally handles class methods by tracking the class context, producing qualified names such as `DataFrame:merge` that accurately reflect the function’s location in the codebase.

Backward Compatibility Detection: Our AST based miner includes logic to identify potentially backward compatible signature changes. When the new parameter types form a subtype relationship with the old types and the function body remains structurally unchanged, the tool flags these as likely non breaking changes. For example, if a function’s parameter annotation changes but the implementation remains the same, this may indicate a documentation update rather than a behavioral change. While this subtype analysis is currently basic, it represents a step toward automatically distinguishing breaking from non breaking changes.

Comprehensive Test Discovery: Our test discovery combines pattern based search with structural validation. We use git grep with carefully crafted patterns to find test files that reference changed functions through various means including direct calls, qualified calls, imports, and string references. We then validate candidates using AST analysis to confirm that the test actually calls the function, reducing false positives from incidental name matches.

No External Dependencies: The tool requires only standard Python libraries and git. There is no need for specialized static analysis frameworks, type checkers, or runtime instrumentation. This simplicity reduces adoption barriers and makes the tool easy to deploy across different development environments.

Impact Based Prioritization: Rather than simply listing affected test files, our ranking system helps developers focus their attention on the most critical changes. Tests that are referenced more frequently across changes, or that occupy higher positions in the test hierarchy, receive higher importance scores. This prioritization is particularly valuable in large projects where a single change might affect dozens of test files.

5.2 Limitations and Threats to Validity

5.2.1 Evaluation Challenges

Ground Truth Assumption: Our evaluation assumes that when developers modify a function signature, they update the relevant test files in the same commit. However, this assumption is often violated in practice. Developers may update tests in subsequent commits, defer test updates to separate pull requests, or neglect to update certain tests altogether. As a result, our ground truth data may be incomplete, and historical validation may be noisy due to these incomplete test updates.

Strict Verification Trade offs: Our call site verification checks whether the function name appears in the changed lines of test files. However, developers may modify tests in ways that do not directly reference the function name, such as updating expected values or modifying fixtures. These legitimate test updates would be missed by strict verification. On the other hand, loose verification that only checks

if a test file was modified may include unrelated changes, introducing noise in the opposite direction.

Selection Bias in Repositories: Our evaluation focuses on scikit-learn and pandas, which are mature, well maintained projects with extensive test suites and strong development practices. Results on these repositories may not generalize to smaller projects, projects with poor test coverage, or projects with different coding conventions. The high density of type annotations in these projects also favors our signature detection approach.

5.2.2 Change Detection Scope

Over Detection of Changes: Our tool detects all function signature changes where type annotations differ between the old and new versions. However, not all such changes require test updates. Many signature changes are backward compatible and do not affect existing callers.

Type Widening and Backward Compatibility: A common pattern in Python development is type widening, where a parameter type is expanded to accept additional types. For example, changing a parameter from `int` to `int | None` typically does not break existing callers that were passing the original narrower type. While our miner attempts to identify such cases by checking if the function body remains unchanged, this heuristic is imperfect. The subtype analysis is currently limited to exact type equality and the `Any` type, and does not understand the full Python type system including generic types, protocols, or structural subtyping.

No Automatic Distinction Between Breaking and Non Breaking Changes: We do not perform deep semantic analysis to determine whether a signature change is truly breaking. Determining backward compatibility would require understanding complex subtype relationships, default parameter values, and how the function body handles different input types. Such analysis is complex in Python due to its dynamic nature and would significantly increase the computational cost of our approach.

Limited to Signature Changes: Our tool focuses exclusively on function signature changes and does not detect other types of changes that may affect tests, such as modifications to function behavior without signature changes, changes to global state, or modifications to class hierarchies.

5.2.3 Test Discovery Limitations

Pattern Based Search Approximation: Our test discovery relies on pattern matching through git grep rather than full static analysis. While this approach is fast and works well for common usage patterns, it may miss indirect references to functions. For example, if a function is stored in a dictionary and called via computed key lookup, or if it is accessed through `getattr`, our patterns will not detect the dependency.

AST Parsing Failures: The AST based validation requires syntactically valid Python code. If a test file contains syntax

errors or uses features not supported by the Python version running the tool, the validation will fail and the file may be incorrectly excluded or included.

False Positives from Common Names: Functions with common names like `min`, `max`, `sum`, or `sort` may match many test files that use unrelated functions with the same name. While our qualified name search patterns help mitigate this, false positives can still occur when the function is imported and called without qualification.

5.2.4 Ranking System Validity

Heuristic Based Scoring: Our test importance scoring system is based on heuristics derived from common testing practices, such as weighting tests higher if they appear in shallower directory hierarchies or if they are named with patterns like `test_api` or `test_common`. These heuristics may not align with the actual importance of tests in all projects.

Frequency Bias: Tests that are frequently coupled with changed functions receive higher importance scores. However, high coupling frequency could indicate either that the test is important and broadly applicable, or that it is poorly designed and overly dependent on implementation details. Our scoring does not distinguish between these cases.

5.3 Potential Improvements and Future Work

Several directions could address the limitations discussed above:

1. **Enhanced Subtype Analysis:** Integrating a type checker like `mypy` or `pyright` could enable detection of truly breaking signature changes by analyzing full subtype relationships. Changes that only widen parameter types or narrow return types could be automatically classified as backward compatible and filtered from results.
2. **Learning from Developer Behavior:** Machine learning models could be trained on historical data to predict which signature changes are likely to require test updates. Such models could learn patterns specific to each codebase and potentially improve both precision and recall over our heuristic approach.
3. **Dynamic Validation:** Combining our static approach with selective dynamic execution could validate whether identified tests actually fail after a signature change, providing higher confidence in the results at the cost of additional computation.
4. **Cross Repository Analysis:** Extending the tool to analyze dependencies across multiple repositories would be valuable for library maintainers who need to understand downstream impact on users of their APIs.

6 Conclusion

Function signature changes are a common source of regression risk in large Python codebases, yet manually locating and prioritizing the tests that need attention is costly and

error-prone. In this paper, we presented *Change-Impact-Predictor*, a lightweight, repository-mining tool that (1) detects signature changes using both a fast RegEx-based miner and a more robust AST-based miner, (2) discovers candidate impacted tests via efficient `git grep` patterns with optional AST validation, and (3) prioritizes tests using a composite ranking model based on coupling frequency, structural depth, and test type. By operating entirely through static analysis and standard tooling, our approach remains practical for large repositories and can be integrated into developer workflows for quick feedback after interface-level edits.

We evaluated our approach through historical validation on two widely used projects, `scikit-learn` and `pandas`. Under the STRICT verification setting, the tool achieves strong ranking performance on `scikit-learn` (MRR = 0.9356, Recall@10 = 99.2%), indicating that relevant tests are typically surfaced near the top of the recommendation list. Performance on `pandas` is lower (MRR = 0.5712, Recall@10 = 83.7%), reflecting a more challenging repository setting and suggesting that practical use may benefit from inspecting a larger top-*k* set. Finally, while our current subtype analysis and ranking signals are intentionally lightweight, they point to several clear future directions, including richer compatibility reasoning via type checkers (e.g., `mypy/pyright`), learning-based ranking models that adapt to repository-specific practices, selective dynamic validation for higher-confidence recommendations, and cross-repository impact analysis for library maintainers.

7 Appendices

A Run Change-Impact-Predictor

A.1 Run Miner

To run regex-based miner, simply run python file `parameter_change_miner_regex.py`

```
python3 parameter_change_miner_regex.py \  
--repo ./sklearn-src \  
--path sklearn/ \  
--since 2019-01-01 \  
--max-shas 1000 \  
--max-rows 200 \  
--out ./datasets/sklearn_changes.csv
```

```
python3 parameter_change_miner_regex.py \  
--repo ./pandas \  
--path pandas/core/ \  
--since 2019-01-01 \  
--max-shas 3000 \  
--max-rows 200 \  
--out ./datasets/pandas_changes.csv
```

To run AST-based miner, simply run python file `parameter_change_miner_ast.py`

```
python3 parameter_change_miner_ast.py \  

```

```
--repo ./sklearn-src \
--path sklearn/ \
--since 2019-01-01 \
--max-shas 1000 \
--max-rows 200 \
--out ./datasets/sklearn_changes.csv
```

```
python3 parameter_change_miner_ast.py \
--repo ./pandas \
--path pandas/core/ \
--since 2019-01-01 \
--max-shas 3000 \
--max-rows 200 \
--out ./datasets/pandas_changes.csv
```

A.2 Run Ranking

To get ranking of the mined tests, simply run python file `parameter_change_ranking.py`

```
python3 parameter_change_ranking.py \
--csv ./datasets/sklearn_changes.csv \
--repo-name sklearn\
```

```
--output-dir ./test_rankings \
```

```
python3 parameter_change_ranking.py \
--csv ./datasets/pandas_changes.csv \
--repo-name pandas
--output-dir ./test_rankings \
```

A.3 Run Evaluation

To run final evaluation of ranked tests, simply run python file `evaluate_predictions.py`

```
python3 evaluate_predictions.py \
--repo ./sklearn-src \
--changes ./test_rankings/sklearn_change_rankings.csv \
--tests ./test_rankings/sklearn_test_rankings.csv
```

References

- [1] Lars Gröninger, Beatriz Souza, and Michael Pradel. 2025. ChangeGuard: Validating code changes via pairwise learning-guided execution. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 936–956.
- [2] Siqi Wang, Xing Hu, Xin Xia, and Xinyu Wang. 2025. ActRef: Enhancing the Understanding of Python Code Refactoring with Action-Based Analysis. *arXiv preprint arXiv:2505.06553* (2025).